

Vulnerability Assessment Report

Prepared by: Muhammad Ali

Cybersecurity Task 1 - Future Interns

Table of CONTENTS

1. Executive Summary.....	2
2. Scope and Ethics.....	3-5
3. Target Website Information.....	6-7
4. Methodology and Practical Assessment.....	8-40
5. Remediation Recommendations.....	41-45
6. Conclusion.....	46

1. Executive Summary

This report outlines a read-only vulnerability assessment conducted on the public web application Google Firing Range (<https://public-firing-range.appspot.com/>). The assessment was conducted employing passive reconnaissance and configuration analysis methods to detect common web security vulnerabilities without exploiting the target system. Tools such as OWASP ZAP Passive Scan, Nmap, and browser developer tools were utilised throughout the assessment process.

2. Scope and Ethics

Scope of Assessment

The vulnerability assessment was performed against the public web application Google Firing Range (<https://public-firing-range.appspot.com/>). The assessment boundaries were restricted exclusively to publicly accessible pages and passive security analysis methods. The objective of the assessment was to identify potential web security vulnerabilities without interacting in a harmful or disruptive manner.

2. Scope and Ethics

Allowed Activities

- Public-facing page assessment
- Passive scanning and reconnaissance
- HTTP security header inspection
- Configuration analysis
- Browser-based security inspection
- Service identification employing non-intrusive enumeration techniques

2. Scope and Ethics

Restricted Activities

- Exploitation of vulnerabilities
- Login bypass attempts
- Brute force attacks
- Configuration analysis
- Denial-of-Service (DoS) attacks
- Credential attacks or authentication misuse
- Any action capable of disrupting the target system

3. Target Website Information

Website Name

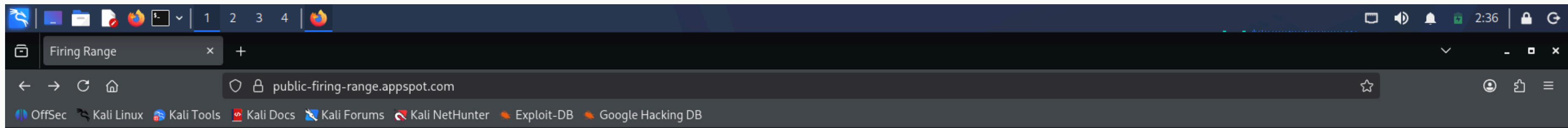
Google Firing Range

Target URL

<https://public-firing-range.appspot.com/>

Website Purpose

Google Firing Range is a publicly accessible web application created to assess and test automated web security scanners. The platform includes deliberately vulnerable components and browser-based security test scenarios for educational and research use.



Version 0.48

What is the Firing Range?

Firing Range is a test bed for automated web application security scanners. Its test cases are not meant to be hard to reach or exercise, as the site can be very easily crawlable. The testbed focuses on detection capabilities, presenting many variants of vulnerabilities and hard-to-detect edge cases. For more details, see the [GitHub page](#).

- [Address DOM XSS](#)
- [Angular-based XSSes](#)
- [Bad JavaScript imports](#)
- [Clickjacking](#)
- [CORS related vulnerabilities](#)
- [DOM XSS](#)
- [Escaped XSS](#)
- [Flash Injection](#)
- [Leaked httpOnly cookie](#)
- [Mixed content](#)
- [Redirect XSS](#)
- [Reflected XSS](#)
- [Remote inclusion XSS](#)
- [Reverse ClickJacking](#)
- [HSTS](#)
- [Tag based XSS](#)
- [URL-based DOM XSS](#)
- [Vulnerable libraries](#)
- [Insecure Third-Party Script Embedding](#)

Figure 1: Google Firing Range target website

4. Methodology and Practical Assessment

The assessment methodology employed passive reconnaissance and read-only vulnerability assessment methods to determine common web security vulnerabilities within the target application. Various security analysis tools were employed to examine exposed services, analyse browser-side security configurations, review HTTP security headers, and detect possible security misconfigurations without exploiting the target system.

Practical Activity 1 - Browser Developer Tools Analysis for Headers, Cookies, and Client-Side Security Issues

Objective

The objective of this activity was to inspect headers, cookies, browser-side behaviour, and client-side security configurations using passive analysis methods.

Tool Used

Browser developer tools are built-in browser utilities employed to examine web requests, response headers, cookies, client-side scripts, and browser security configurations.

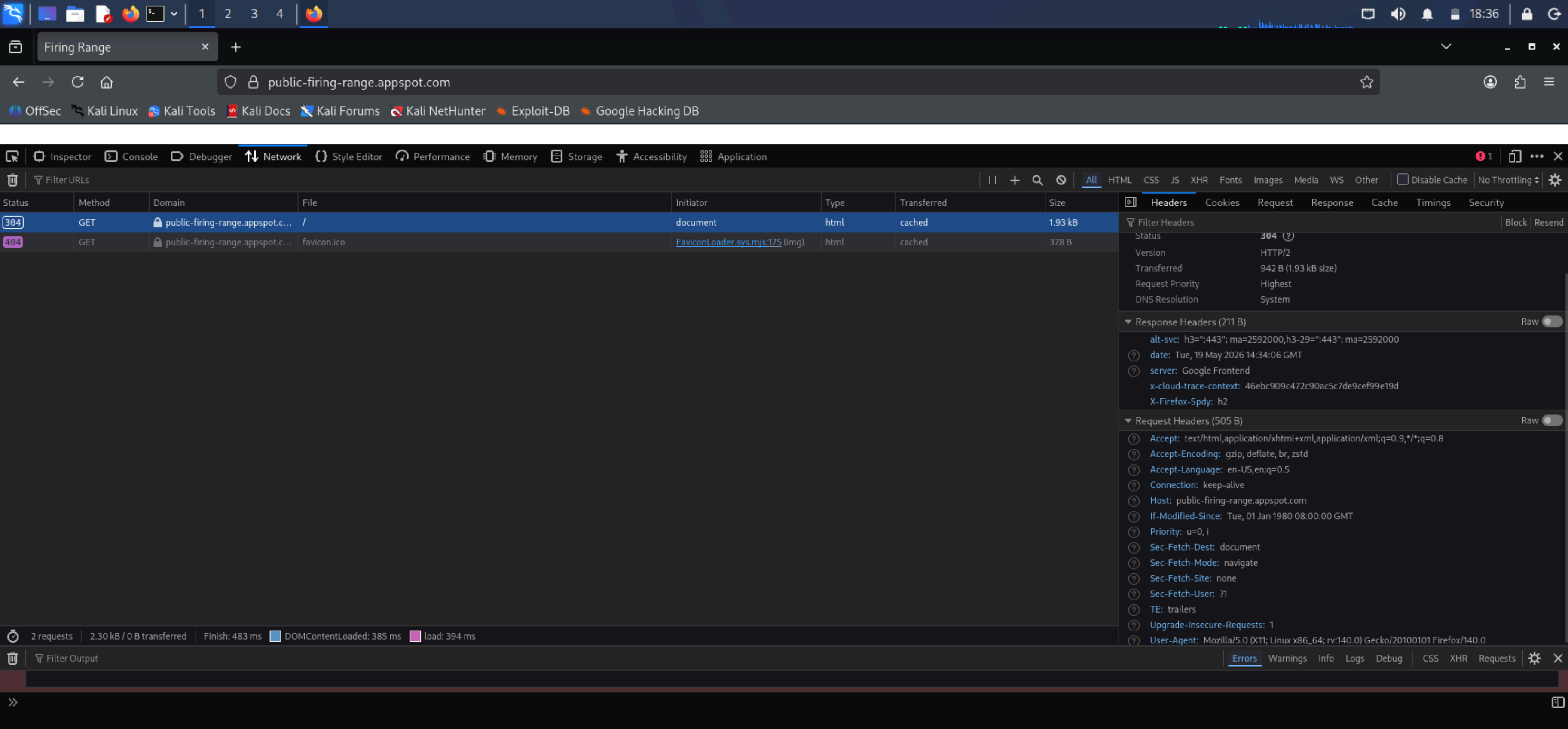


Figure 2: 304 Request Headers

Finding 1 - Exposure of Server and Infrastructure Information Through HTTP Headers

Description

The HTTP 304 revealed infrastructure-related information through response headers, including server: Google Frontend and x-cloud-trace-context.

Evidence

- HTTP Status Code: 304 Not Modified
- server: Google Frontend
- x-cloud-trace-context

Impact

Exposure of backend infrastructure information may help attackers perform reconnaissance and fingerprinting activities.

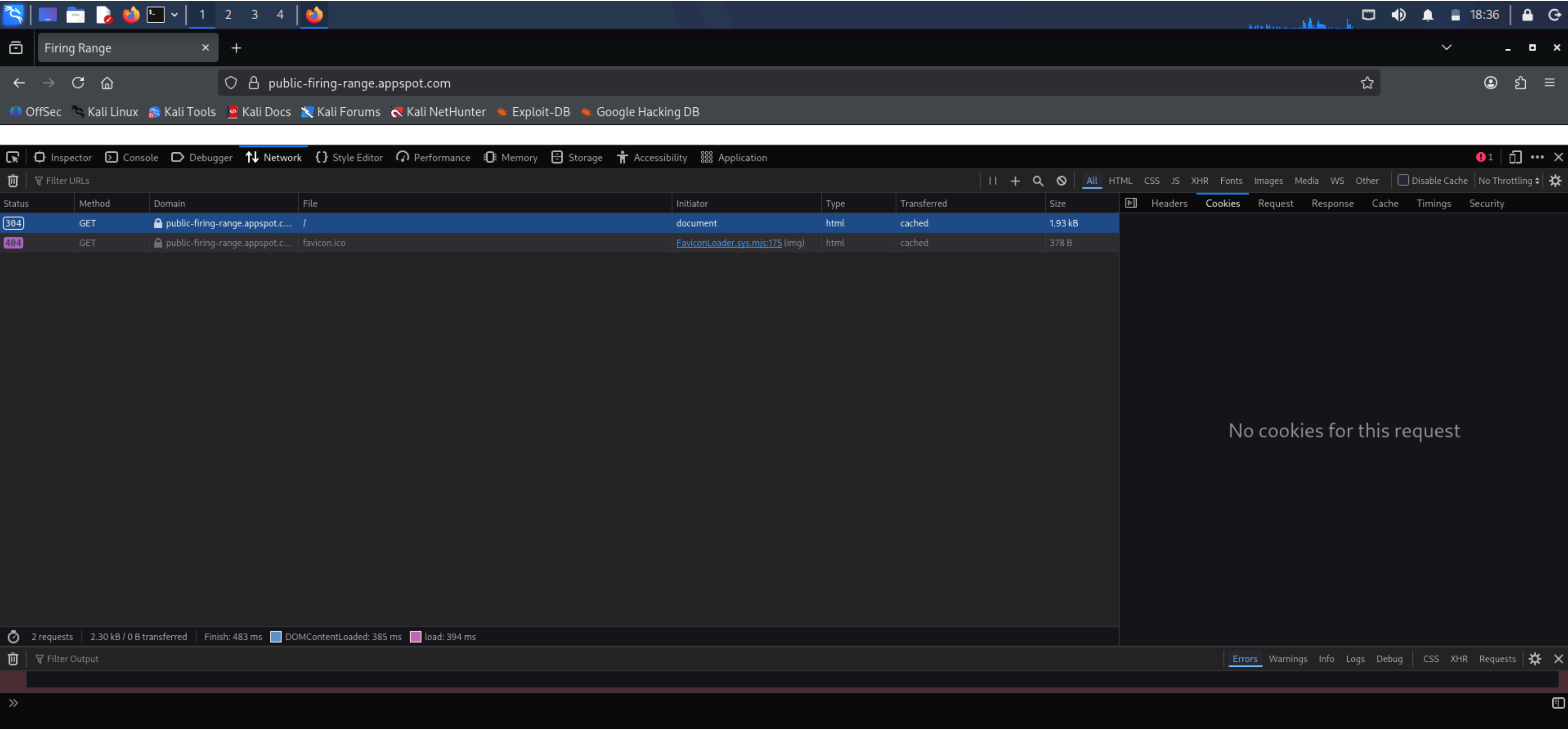


Figure 3: 304 Cookies

Finding 2 - No Cookies Associated With the HTTP Request

Description

The cookies inspection panel indicated that no cookies were sent or stored in the course of the request.

Evidence

“No cookies for this request”

Impact

No session identifiers or sensitive cookie data were disclosed during the observed request.

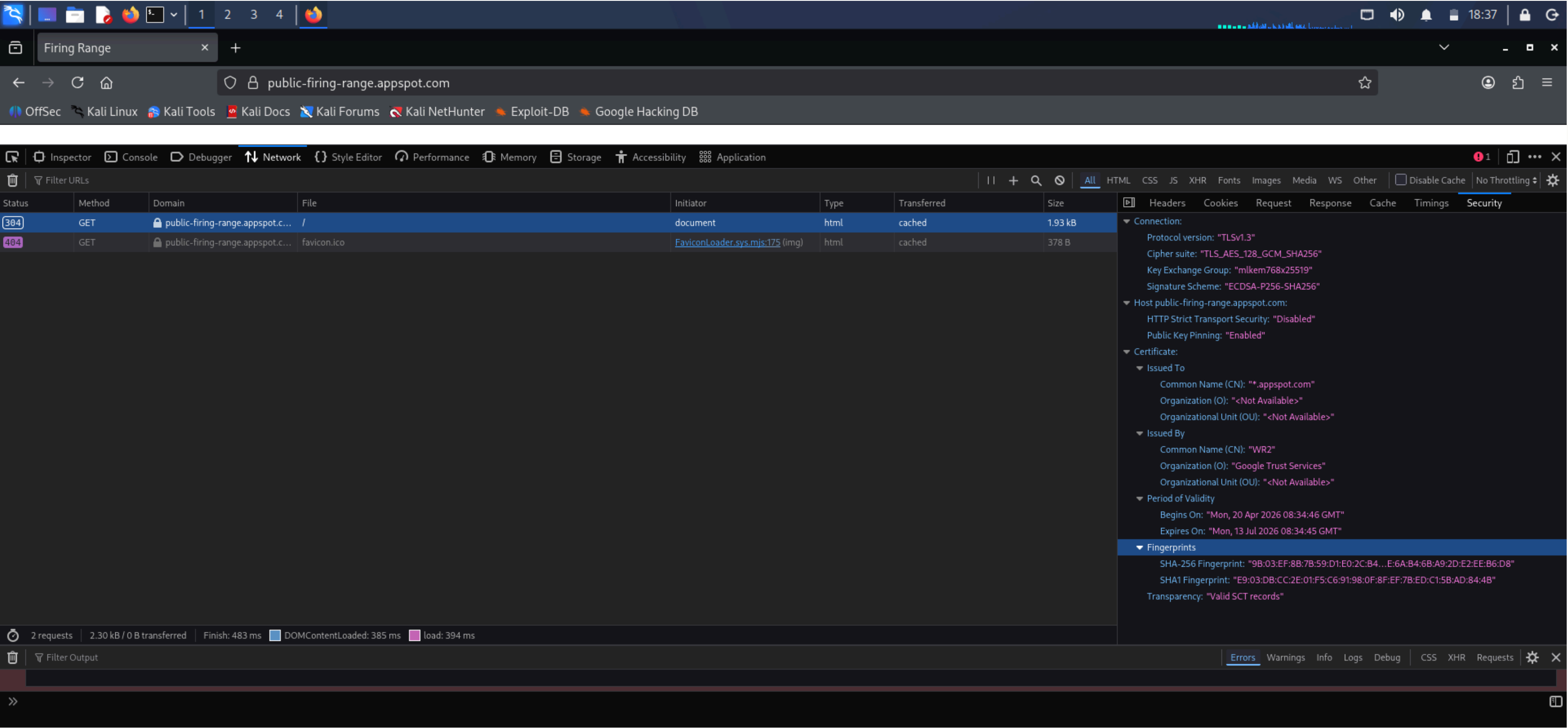


Figure 4: 304 Security Information

Finding 3 - HTTP Strict Transport Security (HSTS) Disabled

Description

The security inspection panel indicated that HTTP Strict Transport Security (HSTS) was disabled for the application.

Evidence

HTTP Strict Transport Security: Disabled

Impact

The absence of HSTS protection may subject users to SSL stripping and protocol downgrade attacks.

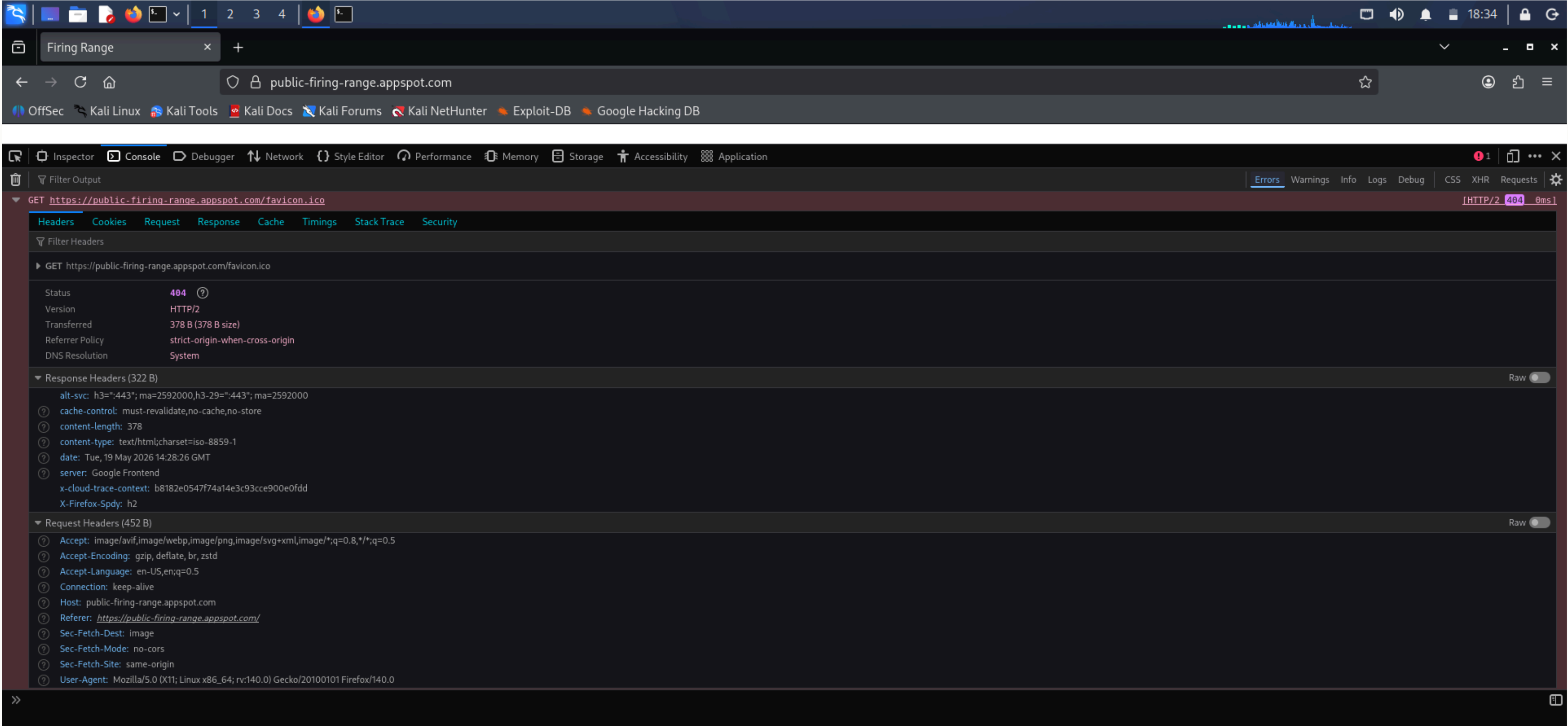


Figure 5: 404 Favicon Request Headers

Finding 4 - Missing Resource Resulting in HTTP 404 Response

Description

The application returned an HTTP 404 response when requesting the favicon.ico resource, showing that the favicon file was unavailable on the server.

Evidence

- Request: /favicon.ico
- HTTP Status Code: 404 Not Found

Impact

Missing resources may signal improper application configuration and can produce unnecessary error responses that help attackers during reconnaissance.

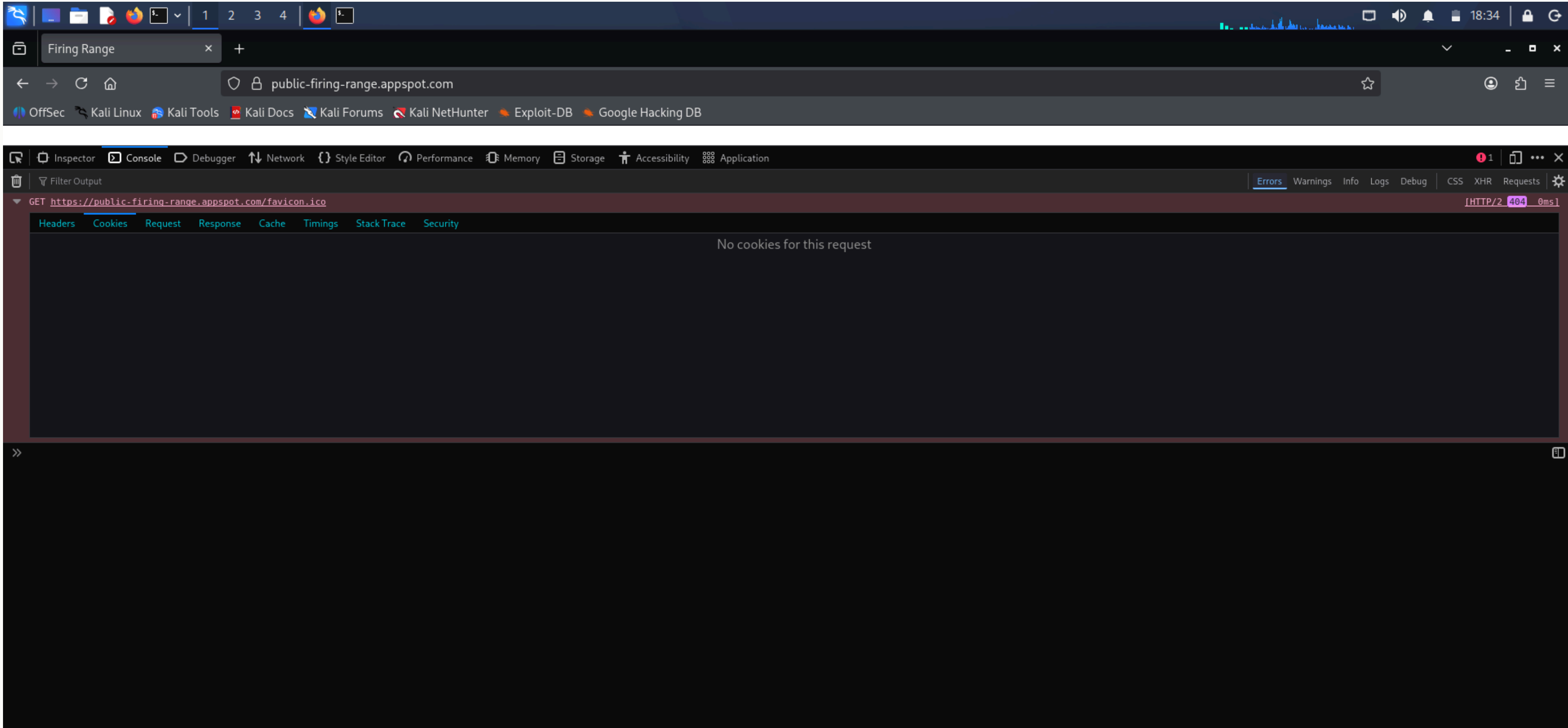


Figure 6: 404 Favicon Cookies

Finding 5 - No Cookies Associated With the Favicon Request

Description

The cookie inspection panel showed that no cookies were sent or stored during the favicon request.

Evidence

“No cookies for this request”

Impact

No session identifiers or authentication cookies were observed during the inspected request.

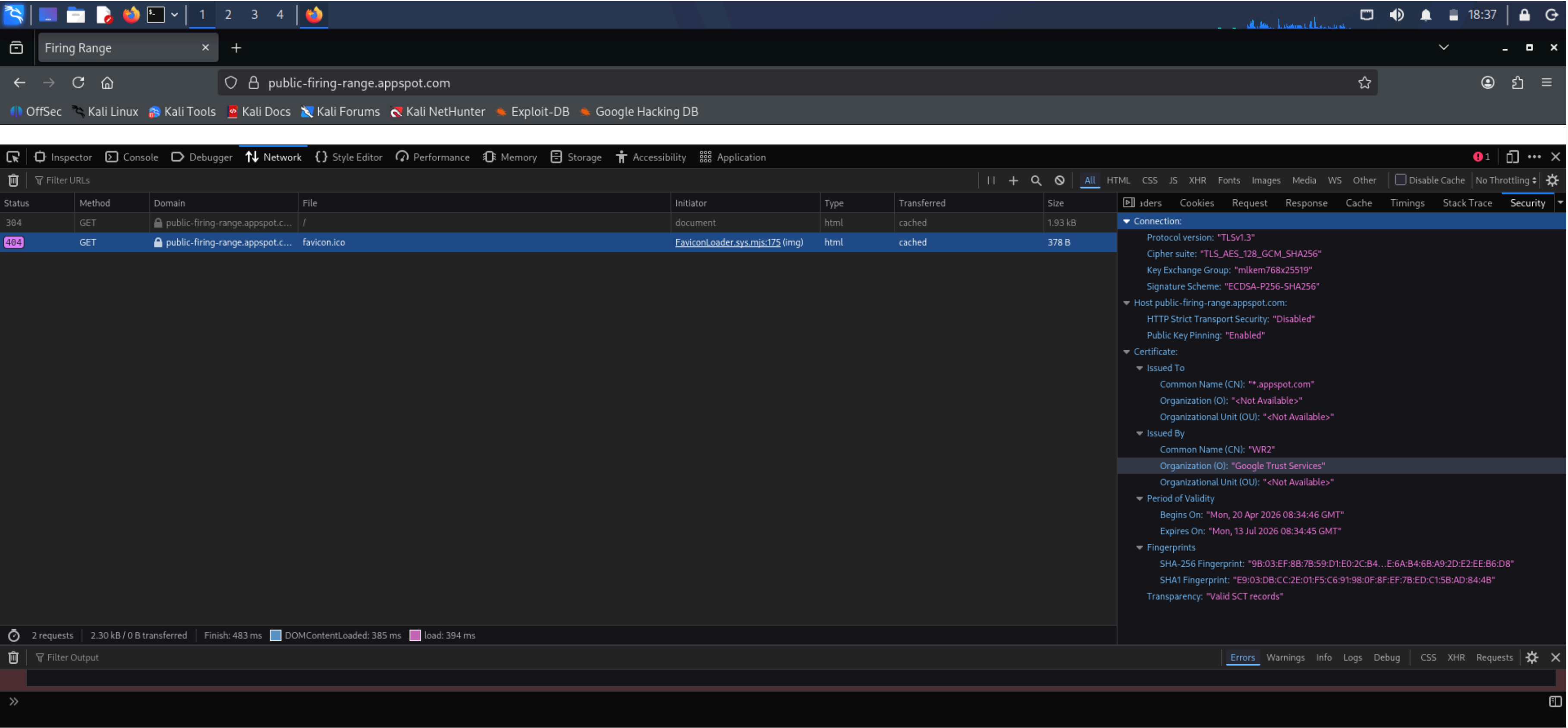


Figure 7: 404 Favicon Security Information

Finding 6 - HTTP Strict Transport Security (HSTS) Disabled

Description

The security inspection panel illustrated that HTTP Strict Transport Security (HSTS) was disabled for the application connection.

Evidence

HTTP Strict Transport Security: Disabled

Impact

The absence of HSTS protection may leave users vulnerable to SSL stripping and HTTPS downgrade attacks.

Practical Activity 2: Passive Vulnerability Assessment Using OWASP ZAP

Objective

The objective of this practical activity was to conduct a passive vulnerability assessment on a public website employing OWASP ZAP without exploiting or attacking the target system. The activity focused on detecting common web security weaknesses, reviewing HTTP security headers, and documenting potential risks in an ethical and read-only manner.

Tool Used

OWASP ZAP is an open-source web application security testing tool used to detect security vulnerabilities, HTTP header weaknesses, and configuration flaws through passive and active scanning methods.

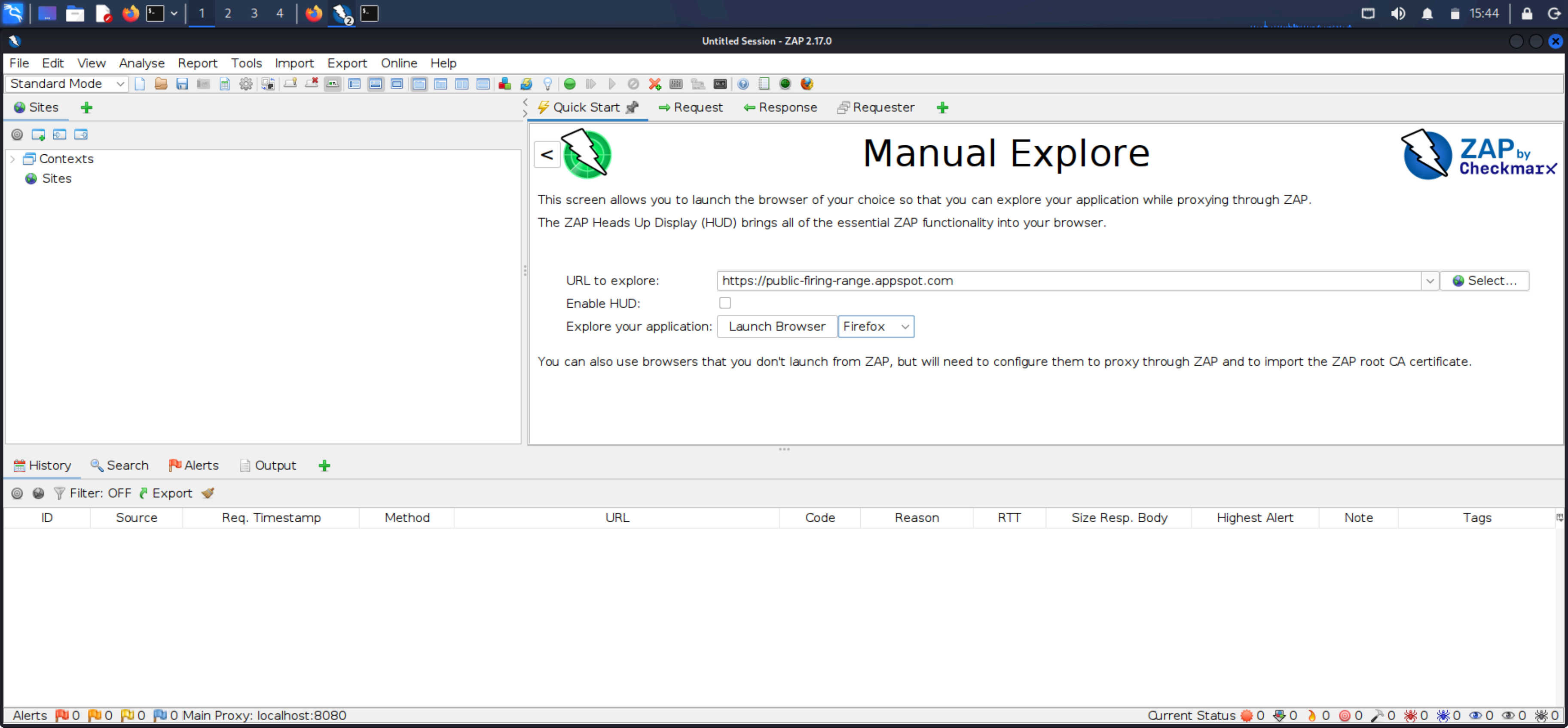
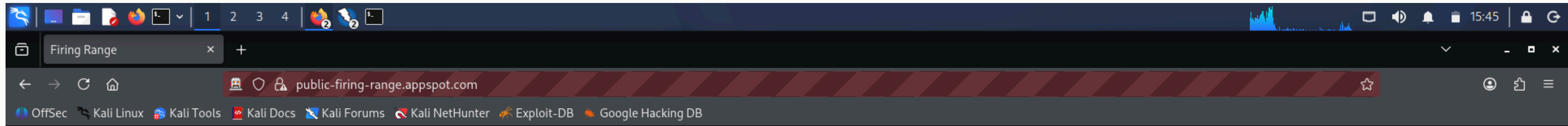


Figure 8: OWASP ZAP Manual Explore Configuration



Version 0.48

What is the Firing Range?

Firing Range is a test bed for automated web application security scanners. Its test cases are not meant to be hard to reach or exercise, as the site can be very easily crawlable. The testbed focuses on detection capabilities, presenting many variants of vulnerabilities and hard-to-detect edge cases. For more details, see the [GitHub page](#).

- [Address DOM XSS](#)
- [Angular-based XSSes](#)
- [Bad JavaScript imports](#)
- [Clickjacking](#)
- [CORS related vulnerabilities](#)
- [DOM XSS](#)
- [Escaped XSS](#)
- [Flash Injection](#)
- [Leaked httpOnly cookie](#)
- [Mixed content](#)
- [Redirect XSS](#)
- [Reflected XSS](#)
- [Remote inclusion XSS](#)
- [Reverse ClickJacking](#)
- [HSTS](#)
- [Tag based XSS](#)
- [URL-based DOM XSS](#)
- [Vulnerable libraries](#)
- [Insecure Third-Party Script Embedding](#)

Figure 9: Public Firing Range Website Opened Through ZAP Proxy

Finding 7 - Missing Content Security Policy (CSP) Header

Description

The OWASP ZAP passive scan detected that the target website does not have a Content Security Policy (CSP) header. CSP is a critical browser security mechanism that helps mitigate attacks including Cross-Site Scripting (XSS) and malicious content injection. The absence of this header may enable unauthorised scripts or resources to run within the browser's environment.

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites +

Quick Start Request Response Requester +

Header: Text Body: Text

Contexts

- Default Context

Sites

- https://public-firing-range.appspot.com
 - GET: /

HTTP/1.1 200 OK

content-type: text/html

last-modified: Tue, 01 Jan 1980 08:00:00 GMT

cache-control: public, max-age=600

vary: Accept-Encoding, User-Agent

x-cloud-trace-context: ef306e707ad3ade204f72a52a390aa96

date: Wed, 20 May 2026 11:45:00 GMT

server: Google Frontend

Content-Length: 1926

Alt-Svc: h3=":443": ma=2592000.h3-29=":443": ma=2592000

<!DOCTYPE html>

<html>

<head>

<title>Firing Range</title>

</head>

<body>

<h1>Version 0.48</h1>

<h1>What is the Firing Range?</h1>

<p>

Firing Range is a test bed for automated web application security scanners.

History Search Alerts Output +

Alerts (5)

- Content Security Policy (CSP) Header Not Set
 - GET: https://public-firing-range.appspot.com/
- Missing Anti-clickjacking Header
 - GET: https://public-firing-range.appspot.com/
- Strict-Transport-Security Header Not Set
 - GET: https://public-firing-range.appspot.com/
- X-Content-Type-Options Header Missing
 - GET: https://public-firing-range.appspot.com/
- Re-examine Cache-control Directives
 - GET: https://public-firing-range.appspot.com/

Missing Anti-clickjacking Header

URL: https://public-firing-range.appspot.com/

Risk: Medium

Confidence: Medium

Parameter: x-frame-options

Attack:

Evidence:

CWE ID: 1021

WASC ID: 15

Source: Passive (10020 - Anti-clickjacking Header)

Alert Reference: 10020-1

Input Vector:

Description:

The response does not protect against 'Clickjacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Alerts 0 2 2 1 Main Proxy: localhost:8080

Current Status 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Figure 11: Missing Anti-Clickjacking Header

Finding 8 - Missing Anti-Clickjacking Protection Header

Description

The scan found that the X-Frame-Options header was not configured on the website. In the absence of this protection, attackers may load the webpage within malicious frames or invisible layers to trick users into selecting unintended elements. This issue elevates the risk of clickjacking attacks.



Finding 9 - Missing HTTP Strict Transport Security (HSTS) Header

Description

OWASP ZAP indicated that the Strict-Transport-Security header was not configured on the web application. HSTS helps ensure secure HTTPS communication between the browser and the server. The unavailability of this header may expose users to insecure communication or downgrade attacks.

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites +

Contexts

- Default Context

Sites

- https://public-firing-range.appspot.com
 - GET: /

Quick Start Request Response Requester +

Header: Text Body: Text

HTTP/1.1 200 OK
content-type: text/html
last-modified: Tue, 01 Jan 1980 08:00:00 GMT
cache-control: public, max-age=600
vary: Accept-Encoding, User-Agent
x-cloud-trace-context: ef306e707ad3ade204f72a52a390aa96
date: Wed, 20 May 2026 11:45:00 GMT
server: Google Frontend
Content-Length: 1926
Alt-Svc: h3=":443": ma=2592000.h3-29=":443": ma=2592000

<!DOCTYPE html>
<html>
<head>
<title>Firing Range</title>
</head>
<body>
<h1>Version 0.48</h1>
<h1>What is the Firing Range?</h1>
<p>
Firing Range is a test bed for automated web application security scanners.

History Search Alerts Output +

Alerts (5)

- Content Security Policy (CSP) Header Not Set
 - GET: https://public-firing-range.appspot.com/
- Missing Anti-clickjacking Header
 - GET: https://public-firing-range.appspot.com/
- Strict-Transport-Security Header Not Set
 - GET: https://public-firing-range.appspot.com/
- X-Content-Type-Options Header Missing**
 - GET: https://public-firing-range.appspot.com/
- Re-examine Cache-control Directives
 - GET: https://public-firing-range.appspot.com/

X-Content-Type-Options Header Missing

URL: https://public-firing-range.appspot.com/
Risk: Low
Confidence: Medium
Parameter: x-content-type-options
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10021 - X-Content-Type-Options Header Missing)
Input Vector:
Description:
The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content

Alerts 0 2 2 1 Main Proxy: localhost:8080

Current Status 0 0 0 0 0 0 0 0 0 0 0 0

Figure 13: X-Content-Type-Options Header Missing

Finding 10 - Missing X-Content-Type-Options Header

Description

The passive scan showed that the X-Content-Type-Options header was missing from the server response. This security header blocks browsers from MIME-type sniffing. Otherwise, browsers may wrongly interpret files and run unintended content.

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites +

Quick Start Request Response Requester +

Header: Text Body: Text

Contexts

- Default Context

Sites

- https://public-firing-range.appspot.com
 - GET: /

HTTP/1.1 200 OK
content-type: text/html
last-modified: Tue, 01 Jan 1980 08:00:00 GMT
cache-control: public, max-age=600
vary: Accept-Encoding, User-Agent
x-cloud-trace-context: ef306e707ad3ade204f72a52a390aa96
date: Wed, 20 May 2026 11:45:00 GMT
server: Google Frontend
Content-Length: 1926
Alt-Svc: h3=":443": ma=2592000.h3-29=":443": ma=2592000

<!DOCTYPE html>
<html>
<head>
<title>Firing Range</title>
</head>
<body>
<h1>Version 0.48</h1>
<h1>What is the Firing Range?</h1>
<p>
Firing Range is a test bed for automated web application security scanners.

History Search Alerts Output +

Alerts (5)

- Content Security Policy (CSP) Header Not Set
 - GET: https://public-firing-range.appspot.com/
- Missing Anti-clickjacking Header
 - GET: https://public-firing-range.appspot.com/
- Strict-Transport-Security Header Not Set
 - GET: https://public-firing-range.appspot.com/
- X-Content-Type-Options Header Missing
 - GET: https://public-firing-range.appspot.com/
- Re-examine Cache-control Directives
 - GET: https://public-firing-range.appspot.com/

Re-examine Cache-control Directives

URL: https://public-firing-range.appspot.com/
Risk: Informational
Confidence: Low
Parameter: cache-control
Attack:
Evidence: public, max-age=600
CWE ID: 525
WASC ID: 13
Source: Passive (10015 - Re-examine Cache-control Directives)
Input Vector:
Description:
The cache-control header has not been set properly or is missing, allowing the browser and proxies to cache content. For static assets like css, js, or image files this might be intended, however, the resources should be reviewed to ensure that no sensitive content will be cached.

Alerts 0 2 2 1 Main Proxy: localhost:8080

Current Status 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Figure 14: Re-examine Cache-Control Directives

Finding 11 - Cache-Control Directives Review

Description

OWASP ZAP produced an informational alert pertaining to cache-control directives. The result indicates that caching behaviour should be examined to verify that sensitive information is not inadvertently stored within browser or proxy caches.

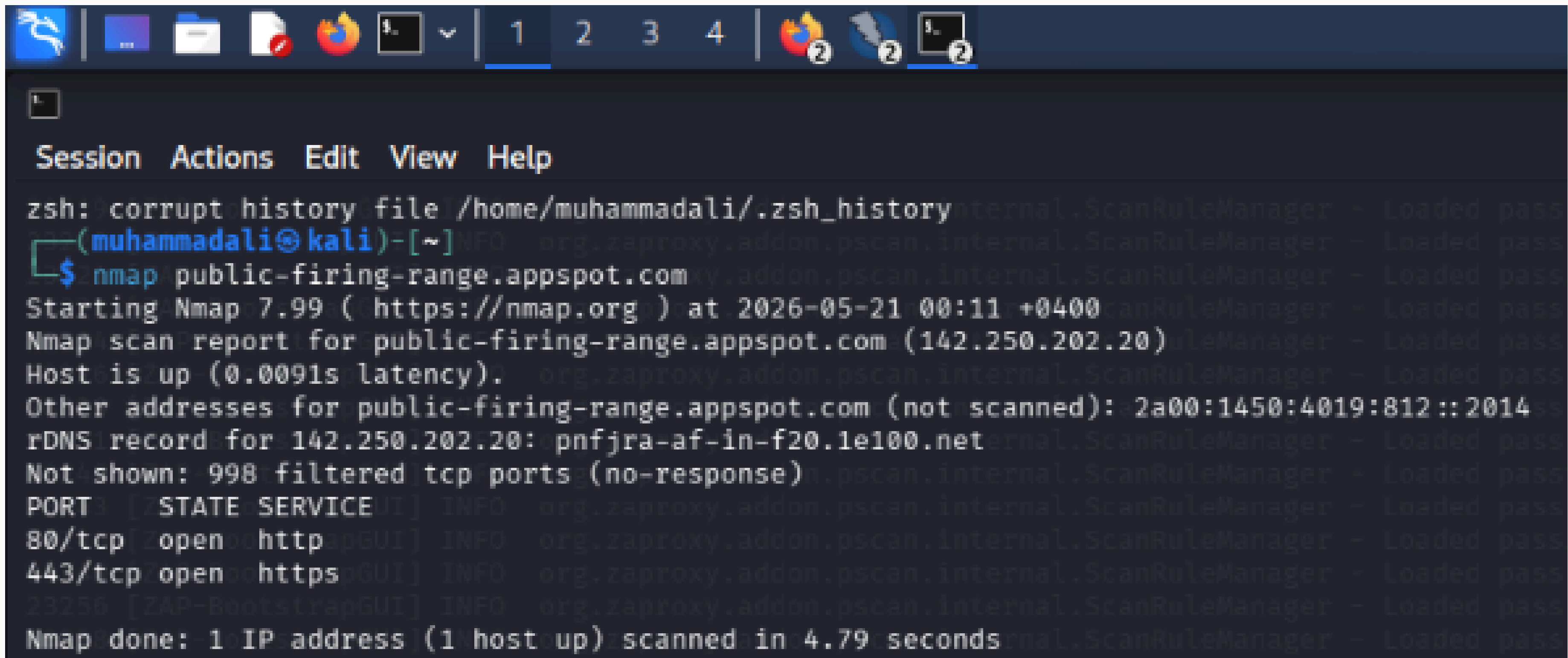
Practical Activity 3: Nmap – Basic Port & Exposure Analysis

Objective

To perform a basic port scanning and exposure analysis of a public web application employing Nmap to identify open ports and exposed services.

Tool Used

Nmap is a network scanning and reconnaissance tool leveraged to identify open ports, exposed services, host availability, and basic network exposure information.



```
zsh: corrupt history file /home/muhammaddali/.zsh_history
(muhammaddali@kali)-[~]
$ nmap public-firing-range.appspot.com
Starting Nmap 7.99 (https://nmap.org) at 2026-05-21 00:11 +0400
Nmap scan report for public-firing-range.appspot.com (142.250.202.20)
Host is up (0.0091s latency).
Other addresses for public-firing-range.appspot.com (not scanned): 2a00:1450:4019:812::2014
RDNS record for 142.250.202.20: pnfjra-af-in-f20.1e100.net
Not shown: 998 filtered tcp ports (no-response)
PORT 80/tcp open http
PORT 443/tcp open https
Nmap done: 1 IP address (1 host up) scanned in 4.79 seconds
```

Figure 15: Basic Nmap scan identifying publicly accessible ports and exposed web services on the target host

Finding 12 - Basic Nmap Port Scan Results

Description

The scan found that ports 80 (HTTP) and 443 (HTTPS) were open and publicly accessible. Furthermore, the scan disclosed the target IP address and reverse DNS information linked to the host. The findings imply that the application is accessible via standard web communication protocols.

```
muhammadali@kali: ~  
Session Actions Edit View Help  
(muhammadali@kali)-[~]  
$ nmap -sV public-firing-range.appspot.com  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 00:12 +0400  
Nmap scan report for public-firing-range.appspot.com (142.250.202.20)  
Host is up (0.0089s latency).  
Other addresses for public-firing-range.appspot.com (not scanned): 2a00:1450:4019:812::2014  
rDNS record for 142.250.202.20: pnfjra-af-in-f20.1e100.net  
Not shown: 998 filtered tcp ports (no-response)  
PORT      STATE SERVICE      VERSION  
80/tcp    open  http         Google Frontend  
443/tcp   open  ssl/https?  
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :  
SF-Port80-TCP:V=7.99%I=7%D=5/21%Time=6A0E15CF%P=x86_64-pc-linux-gnu%r(GetR  
SF:equest,18C,"HTTP/1\0\20404\20Not\20Found\r\nContent-Length:\20272\  
SF:r\nContent-Type:\20text/html;\20charset=UTF-8\r\nDate:\20Wed,\2020\  
SF:\20May\202026\2020:13:00\20GMT\r\n\r\n\n<html><head>\n<meta\20http-  
SF:equiv=\20content-type\20content=\20text/html; charset=utf-8\20>\n<title>  
SF:404\20Page\20not\20found</title>\n<body\20text=#000000\20  
SF:bgcolor=#ffffff>\n<h1>Error:\20Page\20not\20found</h1>\n<h2>The\20r  
SF:equested\20URL\20was\20not\20found\20on\20this\20server\20</h2>\n  
SF:<h2></h2>\n</body></html>\n")%r(HTTPOptions,18C,"HTTP/1\0\20404\20No  
SF:t\20Found\r\nContent-Length:\20272\r\nContent-Type:\20text/html;\20  
SF:charset=UTF-8\r\nDate:\20Wed,\2020\20May\202026\2020:13:00\20GMT\  
SF:r\n\r\n\n<html><head>\n<meta\20http-equiv=\20content-type\20content=  
SF:\20text/html; charset=utf-8\20>\n<title>404\20Page\20not\20found</title  
SF:>\n</head>\n<body\20text=#000000\20bgcolor=#ffffff>\n<h1>Error:\20Pa  
SF:ge\20not\20found</h1>\n<h2>The\20requested\20URL\20was\20not\20f  
SF:ound\20on\20this\20server\20</h2>\n<h2></h2>\n</body></html>\n")%r(RT  
SF:SPRequest,6B0,"HTTP/1\0\20400\20Bad\20Request\r\nContent-Type:\20t  
SF:ext/html;\20charset=UTF-8\r\nReferrer-Policy:\20no-referrer\r\nConten  
SF:t-Length:\201555\r\nDate:\20Wed,\2020\20May\202026\2020:13:00\20  
SF:GMT\r\n\r\n\n<!DOCTYPE\20html>\n<html\20lang=en>\n<\20\20<meta\20char  
SF:set=utf-8>\n<\20\20<meta\20name=viewport\20content=\20initial-scale=1  
SF:,\20minimum-scale=1,\20width=device-width\20>\n<\20\20<title>Error\2  
SF:0400\20\20(Bad\20Request)\20!! 1</title>\n<\20\20<style>\n<\20\20\20\20\2  
SF:0\20*{margin:0;padding:0}html,code{font:15px/22px\20arial,sans-serif}htm  
SF:l{background:#fff;color:#222;padding:15px}body{margin:7%\20auto\200;m  
SF:ax-width:390px;min-height:180px;padding:30px\20\2015px}\20*\20>\20bo  
SF:dy{background:url(//www.google.com/images/errors/robot.png)\20100  
SF:%\205px\20no-repeat;padding-right:205px}p{margin:11px\20\2022px;ov  
SF:erflow:hidden}ins{color:#777;text-decoration:none}a\20img{border:0}me  
SF:dia\20screen\20and\20(max-width:772px)}body{background:none;margin  
SF:-top:0;max-width:none;padding-right:0}}#logo{background:url(//www.goo  
SF:gle.com/images/branding/googlelogo/1x");  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 76.53 seconds
```

Figure 16: Nmap service version detection scan showing exposed services and infrastructure-related information

Finding 13 - Nmap Service Version Detection and Exposure Analysis

Description

The service detection scan found Google Frontend running on port 80 and identified HTTPS services on port 443. The scan also revealed infrastructure fingerprinting information and server response details. Such information exposure may help attackers during reconnaissance and technology identification activities.

Risk Severity	Number of Findings	Related Activities	Related Findings
High	0	None	No high-risk vulnerabilities identified
Medium	6	Activities 1, 2, and 3	Missing CSP Header, Missing Anti-Clickjacking Header, HSTS Disabled, Infrastructure Information Disclosure, Service Exposure Information
Low	4	Activities 1, 2, and 3	Missing X-Content-Type-Options Header, Missing Favicon Resource, Open HTTP Service Exposure, Cache Behaviour Concerns
Informational	3	Activities 1, 2, and 3	No Cookies Observed, Cache-Control Review Alert, Reverse DNS and IP Disclosure
Total	13		Total findings identified during the assessment

Figure 17: Risk Severity Table

5. Remediation Recommendations

The following remediation measures are suggested based on the findings discovered during practical activities 1 to 3, namely browser developer tools analysis, OWASP ZAP passive assessment, and Nmap exposure analysis.

1. Implement Content Security Policy (CSP)

A correctly configured content security policy header should be deployed to mitigate the risk of cross-site scripting (XSS) and malicious script injection threats. Restricting untrusted scripts and external resources can enhance browser-side security protection.

2. Enable Anti-Clickjacking Protection

The web application should implement the X-Frame-Options header or CSP frame-ancestors policy to prevent the website from being placed within malicious frames. This lowers the risk of clickjacking attacks targeting users.

3. Configure HTTP Strict Transport Security (HSTS)

The Strict-Transport-Security header should be enabled to ensure HTTPS communication between clients and the server. Enabling HSTS helps stop SSL stripping and protocol downgrade attacks.

4. Implement X-Content-Type-Options Header

The server should enable the X-Content-Type-Options: nosniff header to block MIME-type sniffing attacks. This guarantees browsers interpret files only based on the declared content type.

5. Reduce Infrastructure Information Disclosure

Server response headers that reveal infrastructure-related information, such as the server type, should be reduced or eliminated where possible. Google Frontend and x-cloud-trace-context should be reduced or eliminated where possible. Reducing unnecessary information disclosure limits reconnaissance capabilities for attackers.

6. Review Cache-Control Configurations

Cache-control headers should be examined carefully to verify that sensitive content is not stored within browsers or intermediary proxy systems. Suitable cache policies should be established based on the sensitivity of application resources.

7. Remove Unnecessary Error Responses

Missing resources like favicon.ico should be addressed to reduce excessive HTTP 404 responses. Effective resource management helps reduce information leakage and minimises reconnaissance indicators available to attackers.

8. Restrict Public Service Exposure

Only necessary services and ports should stay publicly accessible. Periodic exposure analysis and port assessments should be performed to dwindle the attack surface and detect unnecessary services.

9. Perform Regular Security Assessments

Regular checks for security weaknesses, reviews of HTTP headers, and analysis of exposure should be done using tools like OWASP ZAP, Browser Developer Tools, curl, and Nmap. Continuous monitoring assists organisations in determining security vulnerabilities before exploitation occurs.

6. Conclusion

This vulnerability assessment found several security vulnerabilities within the Google Firing Range web application employing passive reconnaissance and read-only analysis approaches. The assessment was performed ethically without attacking the target system and utilised tools like Browser Developer Tools, OWASP ZAP, curl, and Nmap. The findings demonstrated problems such as missing Content Security Policy (CSP), disabled HTTP Strict Transport Security (HSTS), missing anti-clickjacking protection, infrastructure information disclosure, and exposure of publicly accessible web services.